Федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский государственный технический университет имени Н.Э. Баумана»
(национальный исследовательский университет)
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)_____

О т ч е т

по лабораторной работе № 3

Название лабораторной работы: Создание консольного приложения

Дисциплина:

Студент гр. **ИУ6-22Б** _____ **М. Д. Акимов**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О. А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цели работы:

1. Изучение основ разработки консольных приложений в среде .NET.
2. Реализация алгоритмической логики решения квадратного уравнения.
3. Освоение механизмов обработки исключительных ситуаций (try-catch, создание собственных типов исключений).
4. Овладение навыками отладки программного обеспечения (пошаговое выполнение, точки останова, инспекция переменных).

1. Описание предметной области и структуры приложения

Для решения задачи была разработана программа на языке C#, реализующая вычисление корней квадратного уравнения вида ax^2+bx+c=0. В структуре проекта использован статический класс Program, содержащий точку входа Main, а также вложенный класс пользовательского исключения NoRealRootsException.

Диаграмма классов:

Структура приложения представлена ниже, демонстрируя иерархию наследования исключения и архитектуру класса программы.

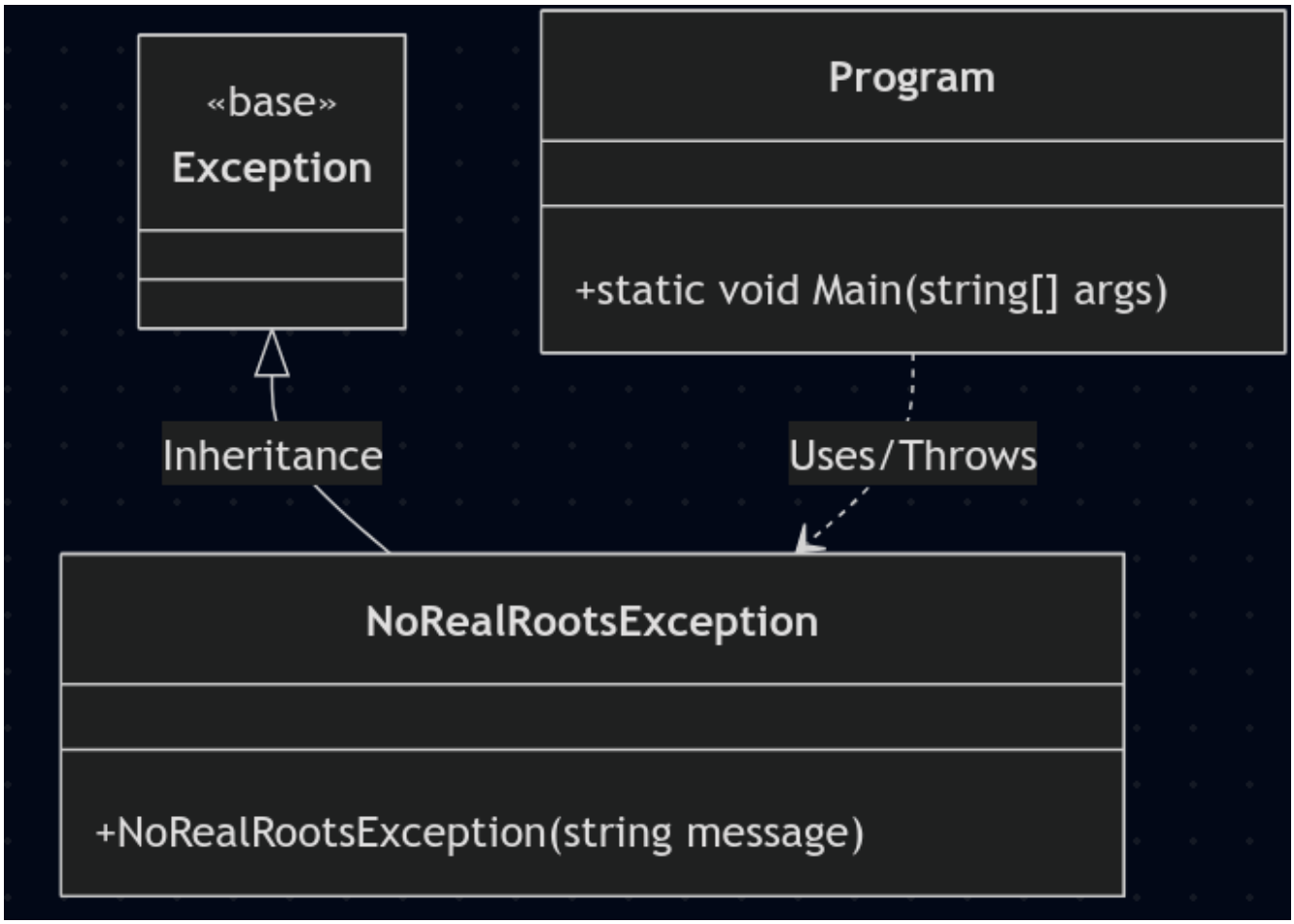


Рисунок 1 - Диаграмма классов

Схема алгоритма (метод Main):

1. **Инициализация:** Вывод приглашения для пользователя.

2. **Ввод данных:** Использование Console.ReadLine() в связке с double.TryParse() для обеспечения отказоустойчивости при вводе нечисловых данных.
3. **Логика вычислений:**
    a. Вычисление дискриминанта D=b^2−4ac.
    b. Если D<0: Выброс NoRealRootsException.
    c. Если D=0: Вычисление единственного корня x=−b/(2a).
    d. Если D>0: Вычисление двух корней по формуле $x_{1,2}$ =(−b±(2a).
4. **Завершение:** Обработка ошибок в блоке try-catch и вывод результата на экран.

2. Реализация программы

Ниже представлен код программы:

```
using System;

namespace ConsoleApp1 { public class Program { public class NoRealRootsException : Exception {
public NoRealRootsException(string message) : base(message) { } }

  private static void Main(string[] args)
   {
      Console.WriteLine("Enter an equation to solve. Example: ax^2 + bx + c = 0");
      Console.WriteLine("Enter the coefficient a:");

      if (!double.TryParse(Console.ReadLine(), out double a))
      {
         Console.WriteLine("Invalid input for coefficient a.");
         return;
      }

      Console.WriteLine("Enter the coefficient b:");

      if (!double.TryParse(Console.ReadLine(), out double b))
      {
         Console.WriteLine("Invalid input for coefficient b.");
         return;
      }

      Console.WriteLine("Enter the coefficient c:");

      if (!double.TryParse(Console.ReadLine(), out double c))
      {
         Console.WriteLine("Invalid input for coefficient c.");
         return;
      }

      double discriminant;

      try
      {
         discriminant = b * b - 4 * a * c;

         if (discriminant < 0)
```

```
        {
            throw new NoRealRootsException("The equation has no real roots.");
        }
        else if (discriminant == 0)
        {
            double root = -b / (2 * a);
            Console.WriteLine($"The equation has one real root: {root}");
        }
        else
        {
            double root1 = (-b + Math.Sqrt(discriminant)) / (2 * a);
            double root2 = (-b - Math.Sqrt(discriminant)) / (2 * a);
            Console.WriteLine($"The equation has two real roots: {root1} and {root2}");
        }
    }
    catch (ArithmeticException ex)
    {
        Console.WriteLine(ex.Message);
    }
    catch (NoRealRootsException ex2)
    {
        Console.WriteLine(ex2.Message);
    }
  }
}
}
```

[

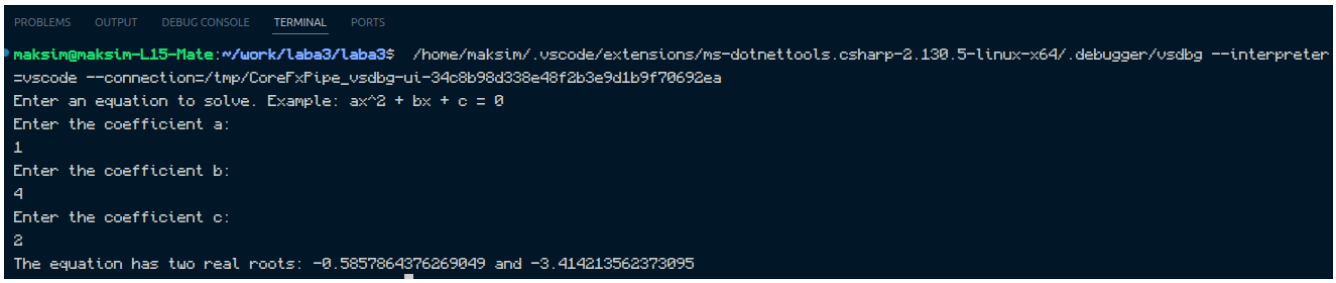


Рисунок 2 - Работа программы при корректных коэффициентах

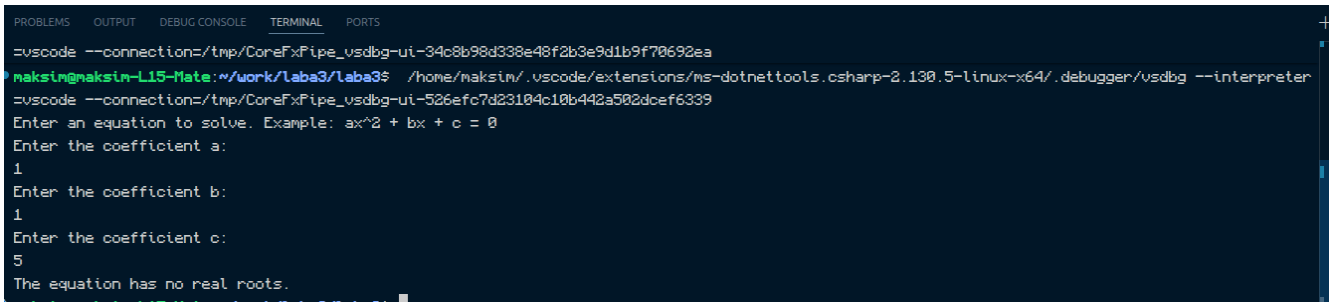


Рисунок 3 - Выброс исключения

3. Диагностика и отладка

В процессе разработки использовались инструменты отладки Visual Studio:

- **Контрольные точки (Breakpoints):** Установлены на строке вычисления дискриминанта для фиксации состояния программы перед выполнением математических операций.
- **Окно "Locals" (Локальные переменные):** Использовалось для отслеживания изменения значений переменных a, b, c после их инициализации.
- **Диагностика компилятора:** Анализировались предупреждения в окне "Error List", что позволило оптимизировать работу с типами данных и избежать потенциальных логических ошибок.



Рисунок 4 - Демонстрация работы отладчика

4. Выводы

В ходе лабораторной работы были изучены основы C# и архитектура консольных приложений. Особое внимание было уделено созданию пользовательских исключений, что позволило реализовать корректную обработку математических ограничений задачи. Изучение средств отладки Visual Studio позволило не только находить ошибки, но и глубоко анализировать поток выполнения программы.